

Reimplementing LoRA: Low-Rank Adaptation of Large Language Models

Rohan Shankar¹ Harshaan Chugh¹

¹Cornell University, CS 4782 — Spring 2026 {rs2656,hsc53}@cornell.edu

Abstract. We reimplemented LoRA [1] from scratch and evaluated it on SST-2 and MRPC with BERT-base and on WikiText-2 with GPT-2 Small. LoRA achieved **93.1%** validation accuracy on SST-2 and **89.9%** validation F1 on MRPC while training only 0.27% of parameters. On WikiText-2, LoRA reached 24.27 perplexity versus 23.60 for full fine-tuning while using 99.5% fewer trainable parameters. Overall, our results support LoRA’s qualitative claim: low-rank task updates can preserve strong performance with dramatically fewer trainable weights.

1. Introduction

Large pretrained language models are expensive to adapt because full fine-tuning updates every parameter and usually requires storing a separate full model copy per task. We reproduce *LoRA: Low-Rank Adaptation of Large Language Models* by Hu, Shen, Wallis, Allen-Zhu, Li, Wang, Wang, and Chen [1]. The paper observes that downstream weight updates often have low intrinsic rank and proposes freezing pretrained weights while learning a low-rank additive update for selected matrices.

Our project reimplements LoRA directly rather than using PEFT or adapter libraries. We test whether the same efficiency-quality tradeoff appears in a smaller reproducible setting: BERT-base [2] on GLUE tasks SST-2 and MRPC [4–6], and GPT-2 Small [3] on WikiText-2 [7].

2. Chosen Result

We aimed to reproduce the paper’s central empirical result: LoRA can match or approach full fine-tuning performance while training only a small number of low-rank parameters. This result is tied to the paper’s low-rank update equation, $\Delta W = BA$, and its benchmark comparisons showing competitive adaptation with far fewer trainable weights [1].

We chose this result because it tests LoRA’s main scientific claim rather than only a surface-level implementation detail. If task adaptation is well approximated by a low-rank update, then LoRA should remain competitive with full fine-tuning, performance should improve with rank until saturation, and the trainable-parameter budget should be much smaller.

3. Methodology

3.1. LoRA Layer Design

For a frozen pretrained weight matrix $W_0 \in \mathbb{R}^{d_{out} \times d_{in}}$, LoRA learns two small matrices $A \in \mathbb{R}^{r \times d_{in}}$ and $B \in \mathbb{R}^{d_{out} \times r}$:

$$h = W_0 x + \frac{\alpha}{r} B A x. \quad (1)$$

Here r is the LoRA rank and α/r scales the update. We initialize A with Kaiming uniform initialization and B to zeros so that $\Delta W = 0$ at the start of training. This preserves the base model’s initial behavior before task-specific learning begins.

We implemented `LoRALinear` for standard `nn.Linear` modules and `LoRAConv1D` for GPT-2’s `Conv1D` attention projection. Both wrappers compute the frozen base path and the low-rank residual path in parallel, then add the scaled residual.

3.2. Injection, Freezing, and Models

Our `apply_lora_to_model()` routine traverses the model graph and replaces modules whose names match target suffixes. For BERT-base, LoRA is inserted into the `query` and `value` projections in all transformer layers. For GPT-2 Small, LoRA is inserted into `c_attn`, GPT-2’s combined attention projection. After replacement, pretrained parameters are frozen; only `lora_A`, `lora_B`, and, for classification, the task head are trainable.

3.3. Datasets, Metrics, and Training Setup

For classification, we evaluate SST-2 using validation accuracy and MRPC using validation F1. For language modeling, we evaluate WikiText-2 using validation perplexity. We vary $r \in \{1, 4, 8, 16\}$ to test the low-rank capacity hypothesis.

For BERT-base experiments, LoRA is applied to `query`, `value`. For GPT-2 Small experiments, LoRA is applied to `c_attn`. Our scripts and notebooks support these configurations and produce the reported tables and figures. Because our experiments used limited Colab compute, this is a smaller-scale reproduction: the goal is to reproduce the qualitative trend—competitive performance with far fewer trainable parameters—rather than exactly match every original paper number.

4. Results & Analysis

Table 1 summarizes the best validation metrics. On SST-2 and MRPC, LoRA outperformed our full fine-

tuning baselines: $r = 8$ achieved 93.1% accuracy on SST-2 and 89.9% F1 on MRPC while training only 0.27% of parameters. This supports the main LoRA claim that constrained low-rank adaptation can be highly parameter-efficient.

Table 1: Best validation metrics. Parentheses show percent trainable parameters.

Task	Method	Params	Metric
SST-2	Full FT	109.5M	85.3% acc
	LoRA $r = 1$	38K (0.04%)	92.0% acc
	LoRA $r = 4$	149K (0.14%)	92.7% acc
	LoRA $r = 8$	296K (0.27%)	93.1% acc
	LoRA $r = 16$	591K (0.54%)	92.7% acc
MRPC	Full FT	109.5M	82.6% F1
	LoRA $r = 1$	38K (0.04%)	88.8% F1
	LoRA $r = 4$	149K (0.14%)	88.6% F1
	LoRA $r = 8$	296K (0.27%)	89.9% F1
	LoRA $r = 16$	591K (0.54%)	89.6% F1
Wiki-2	Full FT	124.4M	23.60 ppl
	LoRA $r = 1$	37K (0.03%)	24.99 ppl
	LoRA $r = 4$	147K (0.12%)	24.43 ppl
	LoRA $r = 8$	295K (0.24%)	24.34 ppl
	LoRA $r = 16$	590K (0.47%)	24.27 ppl

The main discrepancy is that our full fine-tuning baselines underperformed LoRA on the GLUE tasks. We attribute this to overfitting and optimization sensitivity: full fine-tuning updates 109.5M parameters on relatively small classification datasets, while LoRA restricts learning to a lower-dimensional subspace and acts as an implicit regularizer.

On WikiText-2, full fine-tuning remained slightly better: 23.60 perplexity versus 24.27 for LoRA at $r = 16$. This is reasonable because language modeling may benefit from updating more of the model, and our GPT-2 setup only targeted `c_attn`. Still, the gap was small relative to the parameter savings.

The rank ablation supports the low-rank hypothesis. Classification performance improves from $r = 1$ to $r = 8$ and then saturates or slightly declines at $r = 16$. For WikiText-2, perplexity improves steadily through $r = 16$. This suggests that useful rank depends on task type and scale: small classification tasks need few adaptation degrees of freedom, while language modeling benefits from more capacity. LoRA also reduced peak GPU memory by up to 58% on SST-2, reinforcing its practical value.

5. Reflections

This project taught us that parameter-efficient fine-tuning depends on both the mathematical idea and the implementation details. The most important choices were which modules to replace, whether base weights

were truly frozen, how A and B were initialized, and whether the task head remained trainable.

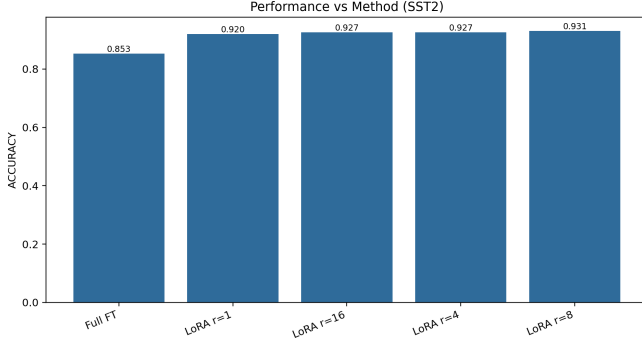
Our results reproduce LoRA’s key trend: low-rank updates can achieve strong downstream performance with orders of magnitude fewer trainable parameters. Given more time, we would run multiple seeds, tune full fine-tuning baselines more carefully, test additional insertion targets such as key/output/MLP projections, and add LoRA weight merging to measure inference latency directly. We would also explore rank-adaptive methods that learn a per-layer rank automatically rather than fixing r as a global hyperparameter.

References

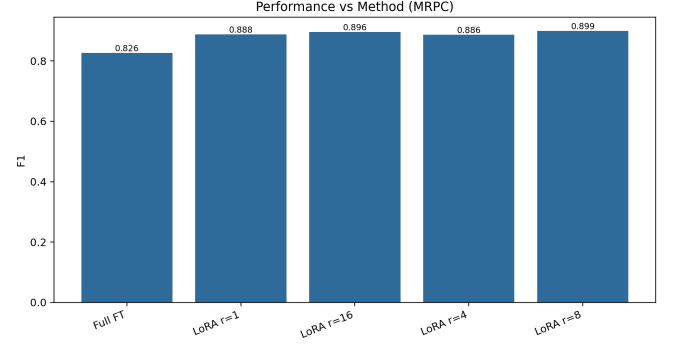
- [1] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *NAACL-HLT*, 2019.
- [3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. OpenAI technical report, 2019.
- [4] A. Wang, A. Singh, J. Michael, F. Hill, O. Levy, and S. Bowman. GLUE: A multi-task benchmark and analysis platform for natural language understanding. *ICLR*, 2019.
- [5] R. Socher, A. Perelygin, J. Wu, J. Chuang, C. Manning, A. Ng, and C. Potts. Recursive deep models for semantic compositionality over a sentiment treebank. *EMNLP*, 2013.
- [6] W. Dolan and C. Brockett. Automatically constructing a corpus of sentential paraphrases. *IWP*, 2005.
- [7] S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models. *ICLR*, 2017.
- [8] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. *NeurIPS*, 2019.
- [9] T. Wolf et al. Transformers: State-of-the-art natural language processing. *EMNLP: System Demonstrations*, 2020.
- [10] Q. Lhoest et al. Datasets: A community library for natural language processing. *EMNLP: System Demonstrations*, 2021.

Figures

All figures are collected here after the 2-page narrative so the text pages remain uninterrupted and clearly within the page limit (figures excluded per the report instructions).

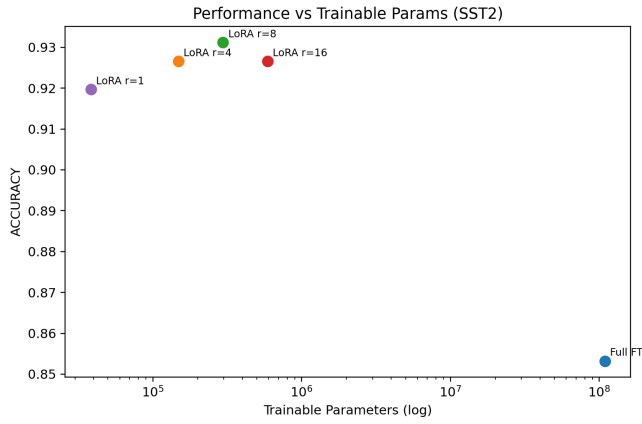


(a) SST-2

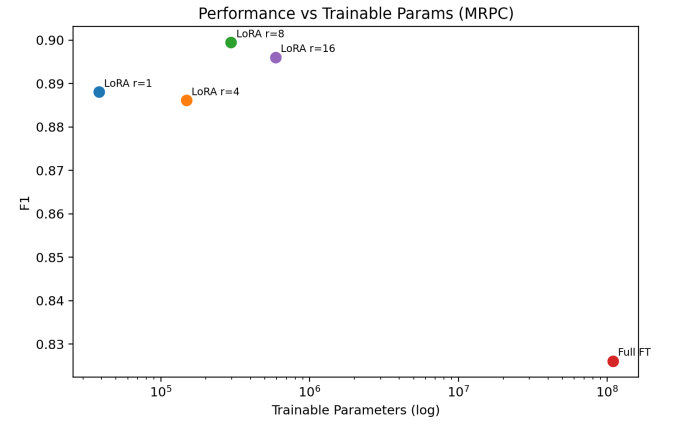


(b) MRPC

Figure 1: Performance comparison between full fine-tuning and LoRA ranks.

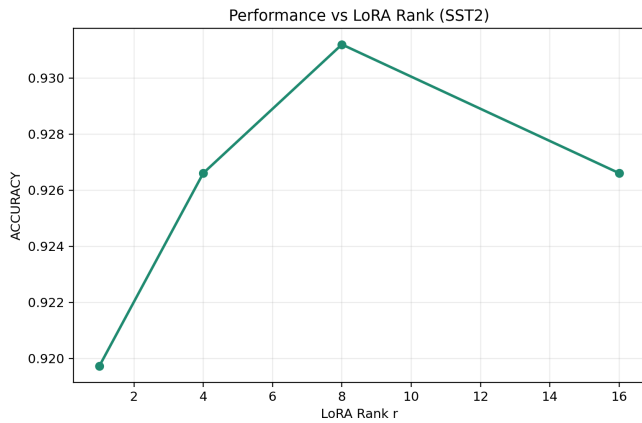


(a) SST-2

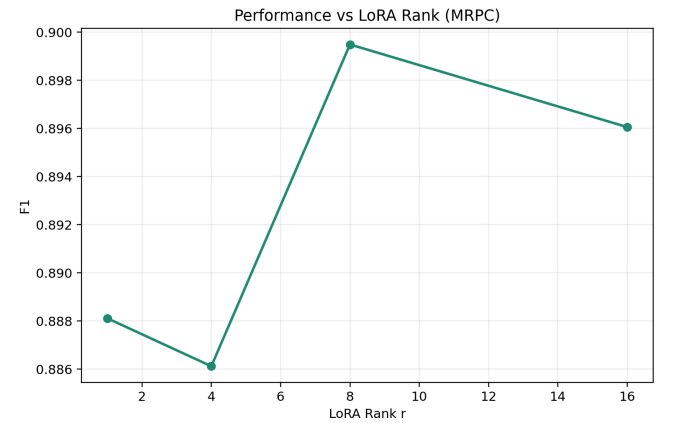


(b) MRPC

Figure 2: Performance versus trainable parameters. LoRA occupies the high-performance, low-parameter region.

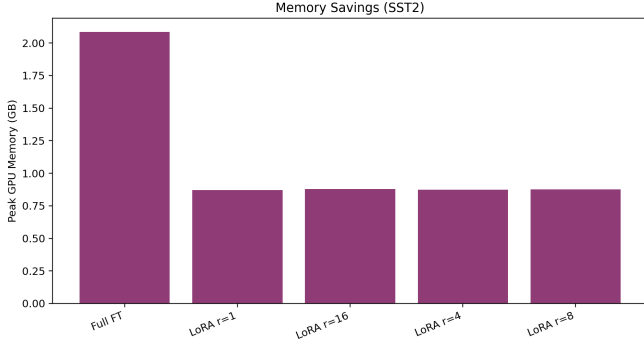


(a) SST-2

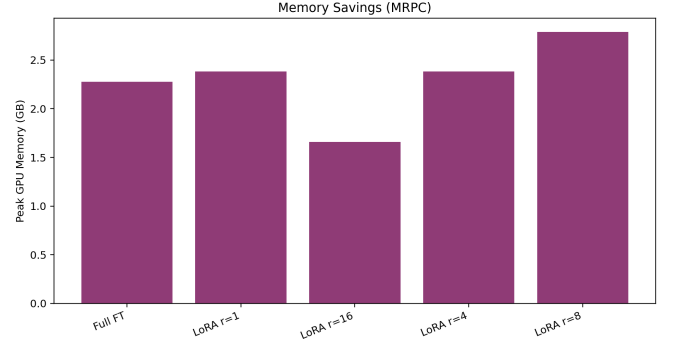


(b) MRPC

Figure 3: Rank ablation. Classification performance improves at low rank and saturates around $r = 8$.

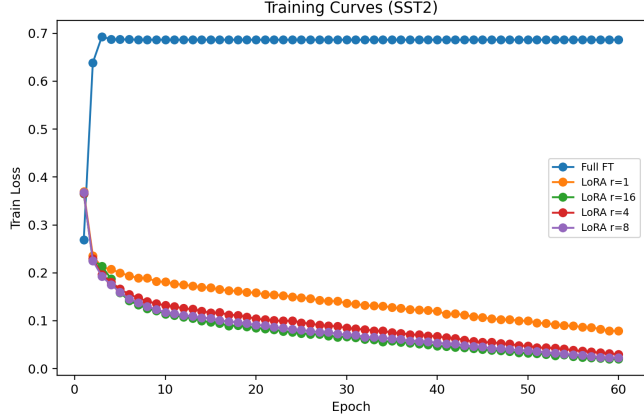


(a) SST-2

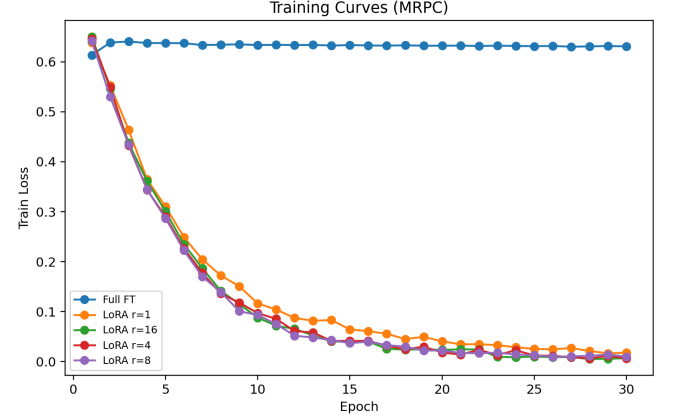


(b) MRPC

Figure 4: Peak GPU memory comparison. LoRA reduces the memory required for adaptation.

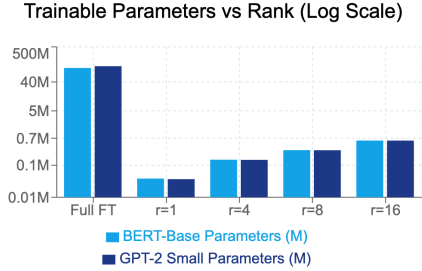


(a) SST-2

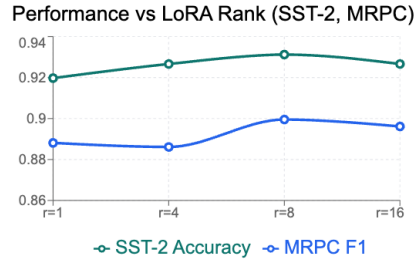


(b) MRPC

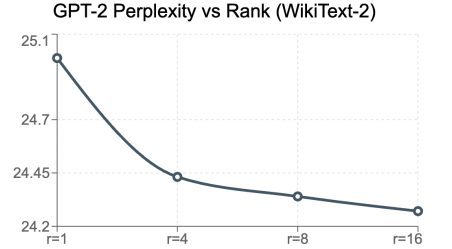
Figure 5: Training curves for full fine-tuning and LoRA variants.



(a) Trainable params vs. rank



(b) GLUE metrics vs. rank



(c) GPT-2 perplexity vs. rank

Figure 6: Cross-model summary of rank, parameter count, and language-modeling perplexity.